

12

Using a UI Component Library and Creating Routes and Navigations

In this chapter, you will learn how to use open source UI libraries built by different Vue.js communities. You will become able to use one of the popular libraries in Vue.js, which will save you from spending countless hours building your components. Then you will set up the navigation and routing of your Vue.js 3 app with best practices in mind.

We will cover the following topics:

- Using a third-party UI component library
- Using other third-party UI libraries
- Adding navigation bars
- Writing page components
- Setting up Vue Router with lazy loading and eager loading

Technical requirements

You need Visual Studio Code to complete this chapter. The finished project of this chapter can be found at this link: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter12/>.

Using a third-party UI component library

What is a **UI component library**? It is merely a library that offers sets of ready-made UI components such as date pickers, forms, cards, navigations, buttons – you name it. Most UI components that you will need in an application are available in a mature UI component library.

What are the benefits of using a third-party UI component library, whether it is open source, free, or paid, that has premium support from the UI component library vendor? The following are the benefits that a UI component library can give you:

- **Saves time:** If creating a robust date picker with a ton of configuration, fewer bugs, and with cross-browser support will take you weeks to finish, using a third-party UI component library is a no-brainer – a UI component library that you can import and try a date picker of the UI component library in less than 4 minutes.
- **Save money:** Since a UI component library saves you countless hours, it saves your client or company money.
- **Reliable:** The top UI component libraries are highly maintained and have very large communities that improve the libraries.
- **Time to market:** If you can ship applications months earlier than your competitors, that can lead to the company's success.

The main takeaway here is not re-inventing the wheel and being conscious of your customer, client, or company's resource spending. A good developer knows how they can contribute to saving resources.

Now that we know why we should use a UI component library, let's include one in a Vue.js project.

Setting up a Vue.js project and installing a UI component library

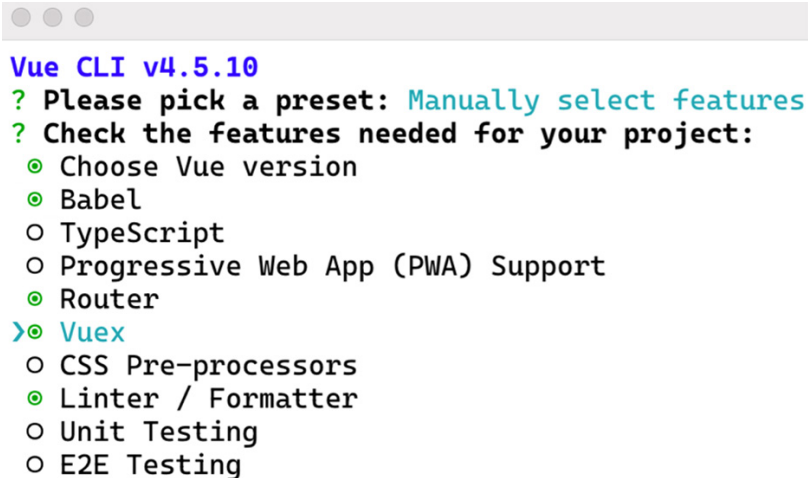
Let's create a new Vue.js project using the following steps:

1. Go to the `presentation` directory of the `Travel` application, and we will run the Vue CLI commands again. Inside the `presentation` folder of the `src` directory, run the following command:

```
vue create vue-app
```

The command will open up the config options for a Vue.js project. Choose *Manually* by pressing the *spacebar* and then *Enter*.

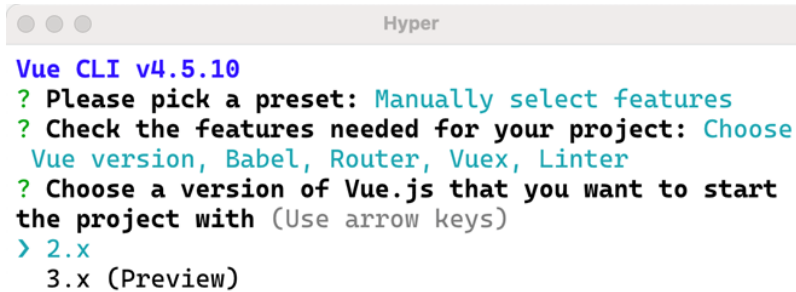
2. Unlike in the last chapter where you enabled the TypeScript, don't enable it this time but enable Router and Vuex instead. FYI, Vuex is for state management. Vuex version 3 and Vuex version 4 have the same interfaces.
3. Keep Babel and Linter enabled. The new Vue.js project's preset should look like *Figure 12.1*:



```
Vue CLI v4.5.10
? Please pick a preset: Manually select features
? Check the features needed for your project:
  ☒ Choose Vue version
  ☒ Babel
  ☐ TypeScript
  ☐ Progressive Web App (PWA) Support
  ☒ Router
> ☒ Vuex
  ☐ CSS Pre-processors
  ☒ Linter / Formatter
  ☐ Unit Testing
  ☐ E2E Testing
```

Figure 12.1 – New Vue.js project's preset

- After selecting the preset, choose 2.x for the version of Vue.js as in *Figure 12.2*:



```

Vue CLI v4.5.10
? Please pick a preset: Manually select features
? Check the features needed for your project: Choose
  Vue version, Babel, Router, Vuex, Linter
? Choose a version of Vue.js that you want to start
  the project with (Use arrow keys)
> 2.x
  3.x (Preview)

```

Figure 12.2 – Vue.js version

- After choosing the 2.x version, the Vue CLI will ask you if you want to use history mode for the router. Select Y or yes by just pressing the *Enter* key on your keyboard.
- Then pick ESLint + Prettier for the linter and formatter. Pick the configuration option with a label that says lint on the **Save** option.
- And lastly, place the config for Babel and ESLint in dedicated config files.

The presentation directory should look like *Figure 12.3* after creating a project using the Vue CLI:



Figure 12.3 – Presentation directory

The `Travel.WebApi` and the `vue-app` projects should be inside the `presentation` folder. Then run the newly created project:

```
npm run serve
```

This command will run the Vue.js project with a logo and a message that says **Welcome to your Vue.js app**. Now, why did we choose JavaScript this time and Vue.js version 2?

Vue.js versions

We chose Vue.js version 2 because 95% of the tutorials on the internet are in Vue.js 2. The majority of the third-party libraries and plugins for Vue.js 3 are not yet compatible with that version. It's sad to say that even though Vue.js 3 was released 4 months ago, most authors of and contributors to third-party libraries and plugins are still migrating their code to the latest Vue.js version. Shout out to all contributors and authors who are working hard and spending their time on small or big commits; you guys make the Vue.js development experience so good.

So that's why we are using Vue.js 2 in this enterprise project instead of Vue.js 3. You should remember this because this is precisely what you should do if there's a new major release from your favorite library or framework. Why?

Because *A*, maybe the majority of external libraries are not yet compatible with the latest version. It's a good idea to create a small proof of concept where typical libraries will work on the newest version of the framework. If other libraries are not compatible with the latest version of the framework, you should wait a number of months for the latest version to mature.

And *B*, there's not enough documentation. Research the latest tooling by answering the following questions:

- How many questions and answers originated from the latest version?
- How many blogs have been published that demonstrate a fully functional application built using the latest version of the framework?
- How many YouTube tutorials have been published for the latest version of the framework?

If you didn't find enough articles or Q-and-As, it means that the framework is not yet mature, and that's a good sign to not use it in an enterprise application for your customers, clients, or companies.

Now let's get back to our Vue.js project and install a UI component library.

Installing Vuetify in a Vue.js application

What is **Vuetify**? Vuetify is a Material Design framework for Vue.js. It has 250,000 weekly downloads on the npm website. If you are not familiar with Material Design, it is a design system created by Google, and they use it in their mobile and web applications. Material Design is so popular that it has also been ported to Angular, React, Svelte, Ember.js, and other JavaScript frameworks.

You can check out all the readily available components on the website, <https://vuetifyjs.com/en/>, or you can go straight to <https://vuetifyjs.com/en/components/alerts/> to see a sample component.

Using or adding the Vuetify library to a Vue application is dead simple. We will use the Vue CLI, particularly the add command of the Vue CLI, which will automatically set up the Vuetify library through a configuration. You will learn more about the configuration I just mentioned in a bit. Now, let get's started:

1. Stop the running Vue.js app and run the following Vue CLI command:

```
vue add vuetify
```

The command will start installing the `vuetify` plugin.

2. Choose the default preset option as in *Figure 12.4*:

```
✓ Successfully installed plugin: vue-cli-plugin-vuetify

? Choose a preset: (Use arrow keys)
> Default (recommended)
  Prototype (rapid development)
  Configure (advanced)
```

Figure 12.4 – Preset options

The default preset provides all framework services with additional baseline values.

3. After adding Vuetify in the project using the Vue CLI, you will see some changes in the project's files. See *Figure 12.5*:

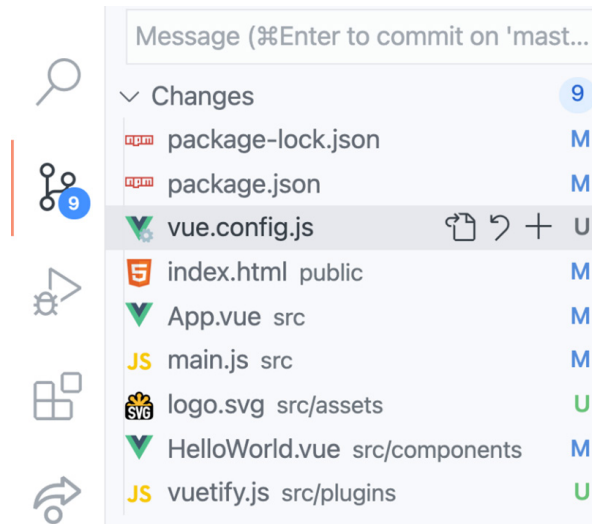


Figure 12.5 – Changes in the project files

The Vue CLI created a `vue.config.js` file, a config file for transpiling Vuetify, and a `vuetify.js` file, which will add Vuetify global functionality to the app. The Vue CLI will also update the `package.json`, `index.html`, `App.vue`, `main.js`, `logo.svg`, and `HelloWorld.vue` files.

4. Rerun the Vue.js application and see that the welcome message changes and the UI, as in *Figure 12.6*:



Figure 12.6 – Welcome to Vuetify

The new welcome message means that Vuetify is fully installed and working correctly in the Vue.js application.

Before we go on to building our navigation bar, let's check out our other UI component library options in the next section.

Other third-party UI libraries

Vuetify is not the only UI component library that you can use in Vue.js. There are other useful UI component libraries too. Here are some libraries that you should check out:

- **BootstrapVue:** This is a popular library for Vue.js that lets you design components using the Bootstrap design system. It has 250,000 weekly downloads on the npm website. You can check it out at <https://bootstrap-vue.org/>.

Figure 12.7 shows BootstrapVue buttons that you can easily use in the Vue.js application:

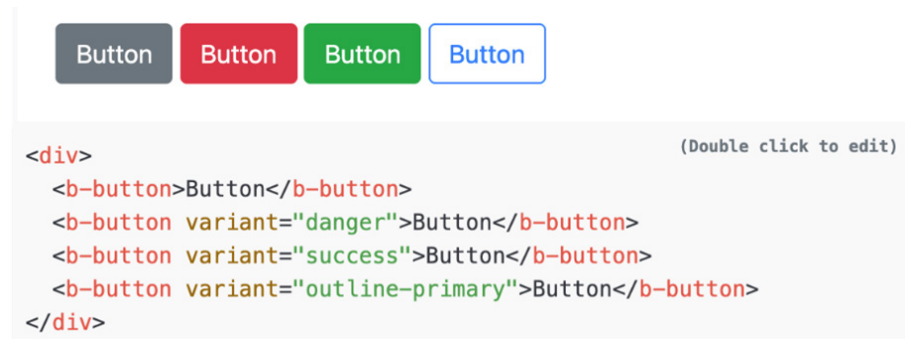


Figure 12.7 – BootstrapVue's button components

- **Buefy:** This is also a popular library for Vue.js, which uses the design system of Bulma and has 45,000 weekly downloads on the npm website. You can check it out at <https://buefy.org/>.

Figure 12.8 shows Buefy buttons that you add to your Vue.js application:

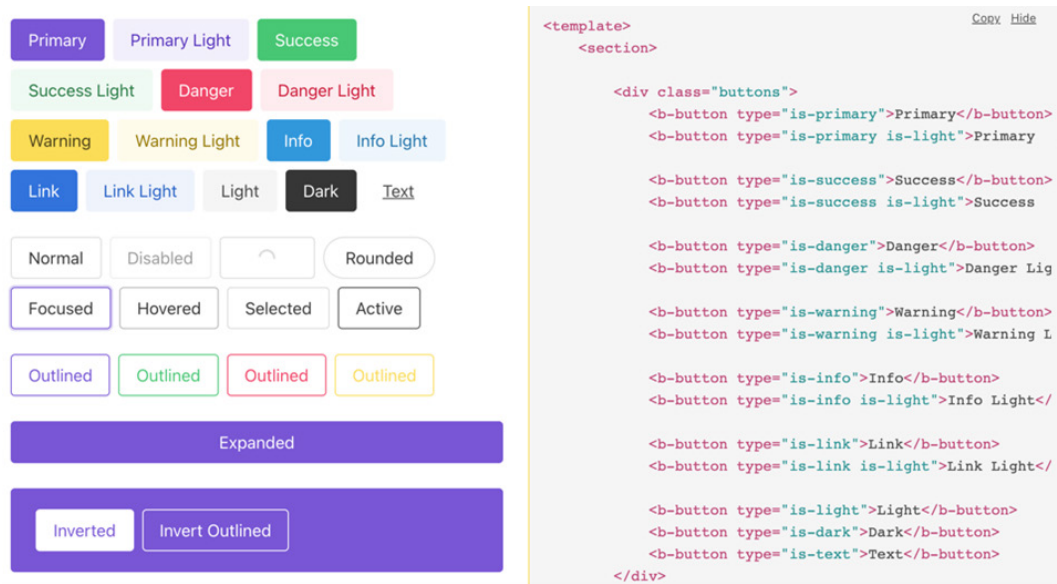


Figure 12.8 – Buefy's button components

- **PrimeVue:** This is a UI component library based on PrimeFaces designs, which is also available in React, Angular, and Java web applications. You can check it out at <http://primefaces.org/primevue/>.

Figure 12.9 shows PrimeVue buttons that you can also easily use in the Vue.js application:

Severities



Raised Buttons



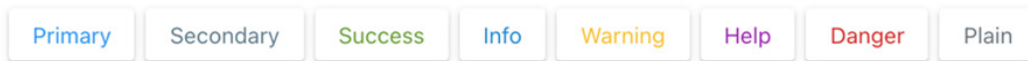
Rounded Buttons



Text Buttons



Raised Text Buttons



Outlined Buttons



Figure 12.9 – PrimeVue's button components

- **Quasar:** This is technically not a library but a framework on top of Vue.js. It is also based on Material Design but has its own CLI and can be compiled to create desktop apps using Electron and iOS and Android apps using Cordova or Stencil. I would recommend this if you are planning to reuse your code base for cross-platform development.

The Quasar framework also offers SSR or server-side rendering, which helps search engines index the pages and content of your Vue.js app. Here's the link to the Quasar framework: <https://quasar.dev/>.

Figure 12.10 shows Quasar button designs that you can see on the Quasar website:

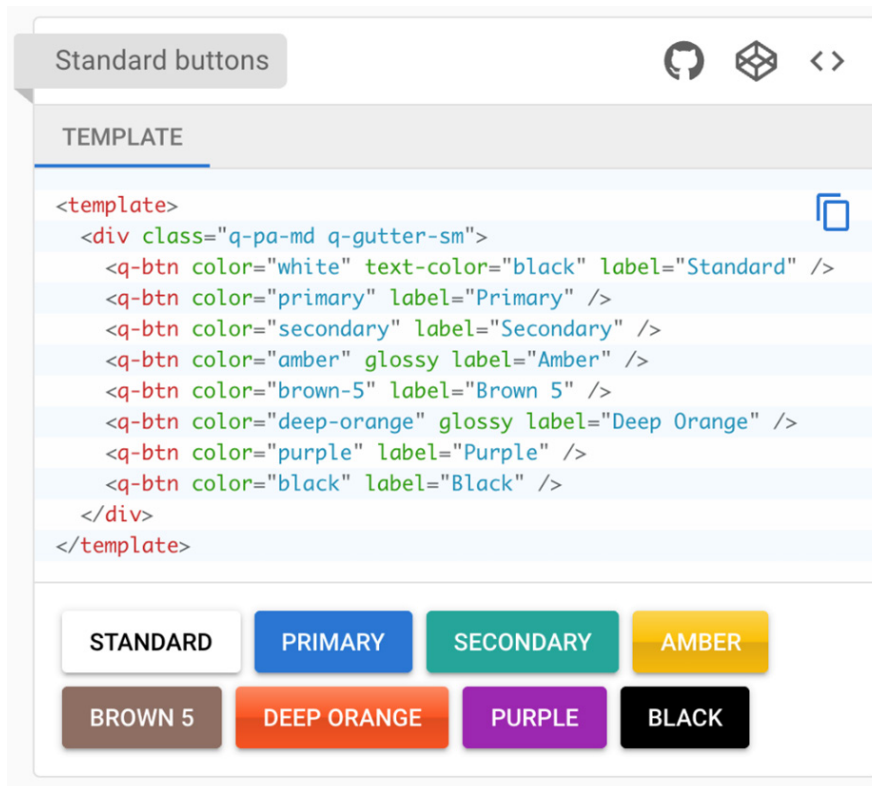


Figure 12.10 – Quasar's button components

Another good thing that I'd like to mention here is that incoming, new Vue.js developers might already be familiar with the UI component libraries that I've listed. This will make the onboarding process faster and will make the freshly hired Vue.js developers productive in a few days.

Now let's go back to our Vue.js project and build the top-bar navigation and sidebar navigation for it.

Adding navigation bars

In this section, we will build a top navigation bar to navigate to the different main pages of our app. We will also make a sidebar inside an admin dashboard page to allow us to navigate between other pages within the admin dashboard. Let's look at the steps:

1. Start by creating a `components` folder in the `src` directory.
2. After creating a `components` folder, create a Vue component file in the `components` folder and name it `NavigationBar`. The path should look like this:
`src | components | NavigationBar.vue`.
3. Now, in the `NavigationBar.vue` file, add the following code:

```
<template>
  <v-app-bar app color="primary" dark>
    <div class="d-flex align-center">
      // for brevity, please see the code in the
      // github repo
    </div>
    <v-spacer></v-spacer>
    <div>
      // for brevity, please see the code in the
      // github repo
    </div>
  </v-app-bar>
</template>

<script>
export default { name: "NavigationBar" };
</script>

<style scoped>
.menu {
  color: white;
  text-decoration: none;
}
</style>
```

The preceding code renders a navigation bar that we will place in our web app's top section. The design was from Vuetify's app bars; I copied the code from Vuetify's documentation and pasted it into VS Code but modified it for our needs. Here's the link to the app bar design: <https://vuetifyjs.com/en/components/app-bars/>. I suggest taking a look at Vuetify's page to familiarize yourself with writing Vuetify components.

You will notice in the code the `v-` prefix – `<v-app-bar>`, for example. This means that the custom HTML is from Vuetify's components.

You will also find the `Vue.js router-link` component in the code. It is used for navigation and will be rendered as a hyperlink tag by default. We set the target route in `to`, `attribute`, or `props` of the `router-link` component.

Next, look at the `<v-btn>` component and check out `:to`, where we are passing an object with the `path` property to set the target route.

The last thing that you might notice here is that we are not importing `defineComponent` from `Vue` because it only works in `Vue.js 3`. We can simply use `export default {}` to share `NavigationBar.vue` with other components.

4. Now that we have our top navigation bar, let's update the `App.vue` component with the following code:

```
<template>
  <v-app>
    <NavigationBar />
    <v-main>
      <router-view />
    </v-main>
  </v-app>
</template>

<script>
import NavigationBar from
  "@/components/NavigationBar";
export default {
  name: "App",
  components: {
    NavigationBar,
  },
}
```

```
};
</script>
```

We are replacing the code of `App.vue` with this code, which imports and uses the `NavigationBar` component.

- Now let's update the `About.vue` component with the following code:

```
<template>
  <div class="about fill-height d-flex justify-center
    align-center">
    <h1>About us 🗺️📱✂️</h1>
  </div>
</template>
```

This is just a message with emojis.

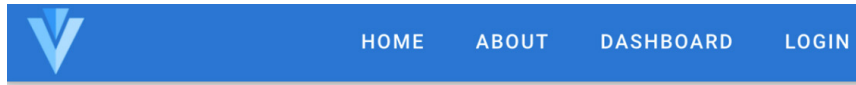
- Let's also update the `Home.vue` component with the following code:

```
<template>
  <div class="home fill-height d-flex justify-center
    align-center">
    <h1>Welcome to Travel Tours 🗺️📱✂️</h1>
  </div>
</template>

<script>
export default {
  name: "Home",
};
</script>
```

We are updating the `Home.vue` component with a simple welcome message.

7. Run the Vue.js application and see the changes in the UI as shown in *Figure 12.11*:



Welcome to Travel Tours

Figure 12.11 – Home page

The screenshot shows the current UI we've developed. It has a top navigation bar and a welcome message on the home page. Most of our functionalities will be on the dashboard and login forms, so let's add more pages in the next section.

Writing page components

Now let's start adding pages to our Vue.js application particularly for admins:

1. In the `views` directory, create a new folder and name it `AdminDashboard`. Create `DefaultContent.vue`, the page for `AdminDashboard`. The `DefaultContent` page will be the default content of the application when a user goes to the `/admin-dashboard` page. Here is the code for `DefaultContent.vue`:

```
<template>
  <div>
    <div>
      <div class="text-h2 my-4">DefaultContent</div>
    </div>
  </div>
</template>
<script>
export default {
  name: "DefaultContent",
```

```
};  
</script>
```

The code is simple enough to show a proof of concept that we can navigate to this page using the `/admin-dashboard` path with text on the browser's screen. We will update this in the upcoming chapters.

2. Create another Vue component and name it `TourLists.vue`. Write the following code in the **TourLists** page:

```
<template>  
  <div>  
    <div>  
      <div class="text-h2 my-4">TourLists</div>  
    </div>  
  </div>  
</template>  
<script>  
  export default {  
    name: "TourLists",  
  };  
</script>
```

The **TourLists** page is for the `/admin-dashboard/tour-lists` path.

3. Create another Vue component and name it `TourPackages.vue`. The following code is for the **TourPackages** page:

```
<template>  
  <div>  
    <div>  
      <div class="text-h2 my-4">TourPackages</div>  
    </div>  
  </div>  
</template>  
<script>  
  export default {  
    name: "TourPackages",  
  };  
</script>
```

The **TourPackages** page is for the `/admin-dashboard/tour-packages` path.

4. Next, create another Vue component and name it `WeatherForecast.vue`. We are going to use the WeatherForecast API in our ASP.NET Core application here, but that will be in the next chapter. So for now, here is the code for the **WeatherForecast** page:

```
<template>
  <div>
    <div>
      <div class="text-h2 my-4">WeatherForecast</div>
    </div>
  </div>
</template>
<script>
export default {
  name: "WeatherForecast",
};
</script>
```

The **WeatherForecast** page will have the `/admin-dashboard/weather-forecast` path.

5. Then, create an `index.vue` component for the AdminDashboard folder. This Vue component will contain the sidebar navigation for the dashboard. Here is the code for the `index.vue` file:

```
<template>
  <v-sheet
    height="100vh" class="overflow-hidden"
    style="display: flex; flex-direction: row;
    justify-content: flex-start"
  >
    <v-navigation-drawer permanent expand-on-hover>
      // For brevity, please see the code in the
      // github repo of this chapter. Thank you.
    </v-navigation-drawer>
    <v-container>
      <router-view />
    </v-container>
```



```
</v-sheet>
</template>
```

The `v-navigation-drawer` component from Vuetify that you can see in the code is a ready-made component that you can copy and paste from <https://vuetifyjs.com/en/components/navigation-drawers/>. Although I copy-pasted the code, I still had to tweak it based on the pages we've created in our application. You can see that we have a small section for the profile UI in the code. There are also links, a link to `/admin-dashboard`, a link to `/admin-dashboard/tour-lists`, a link to `/admin-dashboard/tour-packages`, a link to `/admin-dashboard/weather-forecast`, and a link for the logout functionality.

6. Next is the component name and styling of the `AdminDashboard/index.vue` component:

```
<script>
export default {
  name: "AdminDashboard",
};
</script>
<style scoped>
.link {
  text-decoration: none;
}
</style>
```

We only have a `link` class here to remove `text-decoration`, but we will update this in the upcoming chapters.

Before we finish this section, let's create a new folder inside the `views` directory and name it `Main`. Move the `About.vue` component and the `Home.vue` component to the `Main` folder. Moving the **About** page and the **Home** page to the `Main` folder gives us a better folder structure. The `Main` folder will contain all the pages outside of the dashboard, while the `AdminDashboard` folder will have all the dashboard pages.

Now we are done creating the dashboard pages, let's update the routes file in the next section.

Setting up Vue Router with lazy loading and eager loading

To navigate to the pages we created, we have to register in routes some paths pointing to their respective components. In that case, let's go to the `router/index.js` file to update it:

1. We will update the paths of the **Home** page and **About** page because we'll move them into a new folder, which is `views/Main/`:

```
import Vue from "vue";
import VueRouter from "vue-router";
import Home from "@/views/Main/Home";
import TourLists from "@/views/AdminDashboard/TourLists";
import TourPackages from "@/views/AdminDashboard/
TourPackages";
Vue.use(VueRouter);
const routes = [
  { path: "/", name: "Home", component: Home },
  /* lazy loading through dynamic import() */
  { path: "/about", name: "About",
    component: () => import("@/views/Main/About") },
  { path: "/admin-dashboard",
    component: () => import("@/views/AdminDashboard"),
    children: [ { path: "",
      component: () => import("@/views/
        AdminDashboard/DefaultContent") },
      { path: "weather-forecast",
        component: () => import("@/views/
          AdminDashboard/WeatherForecast") },
      /* eager loading through static import statement
        */
      { path: "tour-lists", component: TourLists },
      { path: "tour-packages", component: TourPackages }
    ] },
  { path: "*", redirect: "/" },
];
const router = new VueRouter({
```

```
mode: "history", base: process.env.BASE_URL, routes,  
  });  
export default router;
```

2. Then we add the routes for the pages we created. The `vue-router` library is responsible for creating routes in our Vue.js project. We are defining all the routes inside the array and passing the array in `VueRouter` while creating an instance of it.

Take a look at the values of the components inside the objects in the array. You will notice that I am showing different ways of importing the components. One is a **dynamic import** where we use `import()`, and the other is **static import**. Now, why do we need to do this? A dynamic import is used here to import components only when the user is navigating to a specific page. The on-demand importing of components or pages based on URLs or the UIs that users see is called lazy loading.

Lazy loading is the way to split code. You will also hear the term code-splitting instead of lazy loading. We do this to not congest the network by downloading files when our web app is loading in the browser. Consequently, a network bottleneck can usually happen if a web application has many files and large files downloading simultaneously in the browser. Lazy loading improves the performance of our web apps.

On the other hand, **eager loading** is what a static import does. The resource files of any pages are downloaded right away when a user visits a web application. If it takes more than 5 seconds for your application to load because of a network bottleneck, that's a bad user experience for web visitors.

You can check out all the APIs of Vue Router at <https://router.vuejs.org/>.

Now let's once again run our application and navigate to the different pages inside the dashboard:

1. Go to the dashboard by clicking the **DASHBOARD** menu. Open up your browser's DevTools, go to the **Network** tab, click **All** for the types, and clear out the logs as shown in *Figure 12.12*:

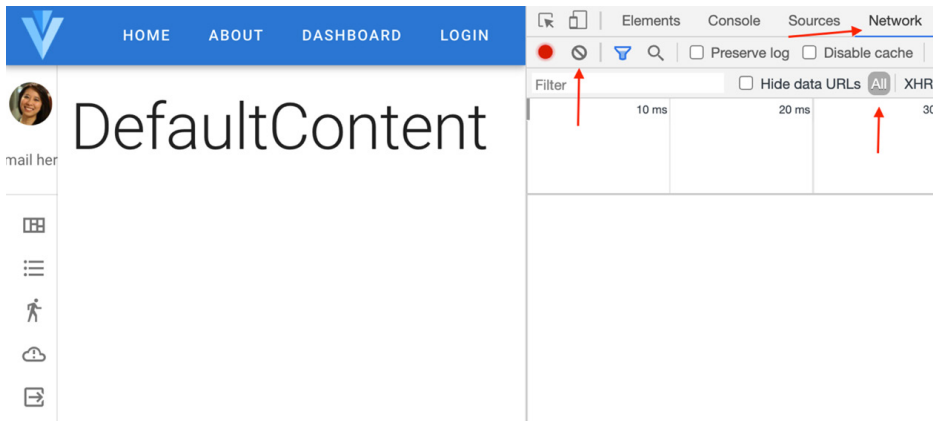


Figure 12.12 – Dashboard default content and DevTools

Make sure to remove the logs to quickly see if a file or files appeared on the screen. We are preparing our web app for a quick demo of how lazy loading works.

2. Hover your mouse on the sidebar so you can see the navigation menus of the sidebar, like in *Figure 12.13*:

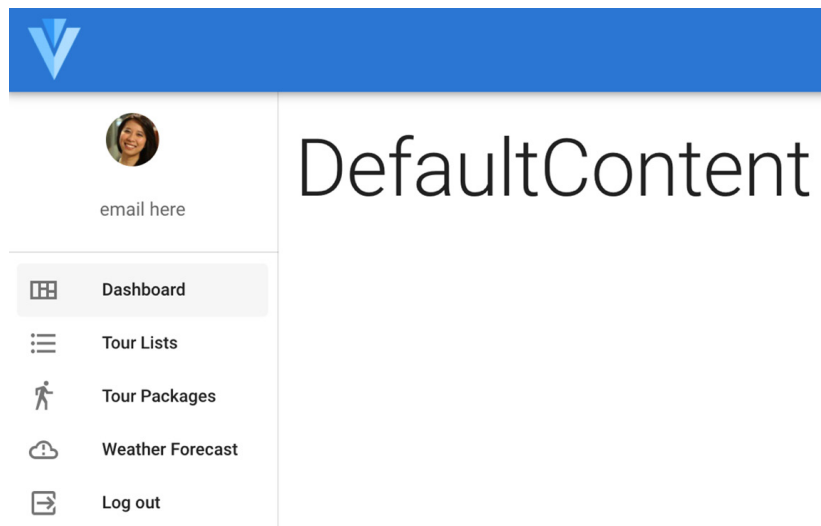


Figure 12.13 – Navigation menus of the sidebar

- Now go to the **Tour Lists** page and **Tour Packages** while looking at the DevTools. There are no downloaded files that you can see in the **Network** tab of the DevTools because those pages are imported using static imports.
- Go to the **Weather Forecast** page and watch out for the script and stylesheet files that will land on DevTools as shown in *Figure 12.14*:

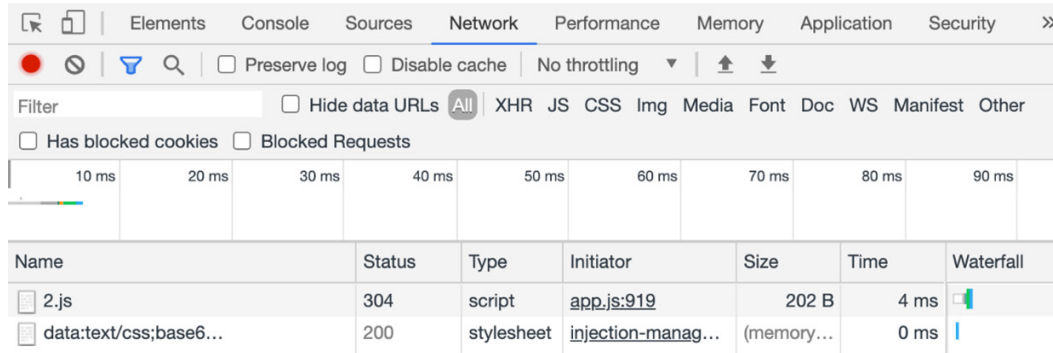


Figure 12.14 – Lazy loaded script file and stylesheet file of the Weather Forecast page

The two files were added as soon as we navigated to the **Weather Forecast** page, using a dynamic import to lazy load the file. Imagine you have more than 50 pages in your Vue.js application. Your application will gain improvements in the loading speed if you lazy load all 50 pages of your application.

And a piece of advice: also apply code-splitting in React, Angular, Svelte, Ember.js, Preact, Solid.js, or any JavaScript framework that you'll face in the future.

Now, before we go to the list of what we have learned and wrap up this chapter, I'd like you to remember the process we followed in starting a project. The following are the steps in building an application that I came up after years of developing web and mobile apps:

- Installing a UI component library or using a ready-made boilerplate from popular GitHub repositories.
- Studying the required pages in the app, knowing how many pages there are, and whether a dashboard admin is needed. You would generally get this information from the UI design team in your company.
- Building the navigations, writing lazy loaded routes, and creating pages with simple welcome messages. Creating these elements right away gives you the overall skeleton of the application.

So that's usually the pattern I use in the early stages of any web or mobile development I do.

Now let's summarize what you have learned in this chapter.

Summary

Great work! We've done so many things in this chapter. You've learned that UI component libraries can help us develop applications faster by not building components from scratch. You have learned that Vuetify, BootstrapVue, Buefy, PrimeVue, and Quasar are some of the UI component libraries with excellent visual designs and optimized for user experience.

You've also learned how to create pages in Vue.js by adding the components in the routes, and how to get from one point of your app to another using the `router-link` component.

And lastly, you have learned how to lazy load pages to improve the loading speed of the web application.

In the next chapter, we will integrate a Vue.js application with ASP.NET Core.